Machine Learning Project

Table of content

Introduction

The crucial first step of this project was to define a theme and a compelling problem statement. Given our shared interest in sports, our initial scope centered on Formula 1. We successfully sourced a comprehensive Formula 1 dataset, which led us to formulate the following research question: "Can we predict the optimal tire strategy for a Grand Prix?"

Unfortunately, the essential data required for this specific prediction was not present in the initial dataset. We attempted to remedy this by utilizing the fastf1 API to gather more detailed information. However, as the API only supports data from 2018 onwards, this attempt resulted in a very limited dataset of less than 500 rows, deemed insufficient for robust training of machine learning models. This prompted our first change in the subject matter.

Our second search for new data led us to a dataset containing 10,000 synthetic entries with information on Spotify users. We then developed the problem statement: "Can we predict whether a user will uninstall the application (i.e., predict customer churn)?" After preliminary work, we discovered that the data had been generated in a completely random way, making it impossible to create a viable predictive model.

Nevertheless, the central idea of predicting customer loyalty remained highly relevant. We now have a realistic and actionable dataset about Bank Customers, as well as a definitive and viable problem statement, which will form the core of our analytical project.

Note: All our work with the Formula 1 dataset, the fastf1 API, and the Spotify dataset will be available in an appendix.

Business case: Predicting bank Customer Churn

Our goal is to predict which customers are likely to leave the bank, in order to reduce churn and improve retention efficiency. We can define the problem as follows: Current customer retention efforts are extensive, costly, and not based on predictive data analysis. Our solution is to develop a machine learning model to identify customers who are most likely to close their accounts. The results will allow the marketing team to focus on those customers with personalized retention actions.

Here is our approach: Use data about customer demographics, transactions, product usage, and service interactions to train and validate a machine learning model with clear explanations for each prediction. Then, we could integrate the model’s predictions into the bank’s marketing and customer management systems. The expected impact of this project is to reduce customer churn rate, increase efficiency and return on investment of retention campaigns.

Description of our dataset and the source

Our dataset is Kaggle dataset. This dataset is a synthetic dataset, originally created for an IBM Hackathon. ‘Exited’ is the binary target. There are 14 features in this dataset and 10,000

lines. After some research we found out that this dataset was a sample of a larger dataset of 165,000 lines, which became our final Dataset[1].

In these 14 features, 3 are **categorical**:

- Surname
- Geography (i.e. Country)
- Gender

8 are **numerical:**

- RowNumber / id
- CustomerId
- CreditScore
- Age
- Tenure
- Balance
- NumOfProducts
- EstimatedSalary

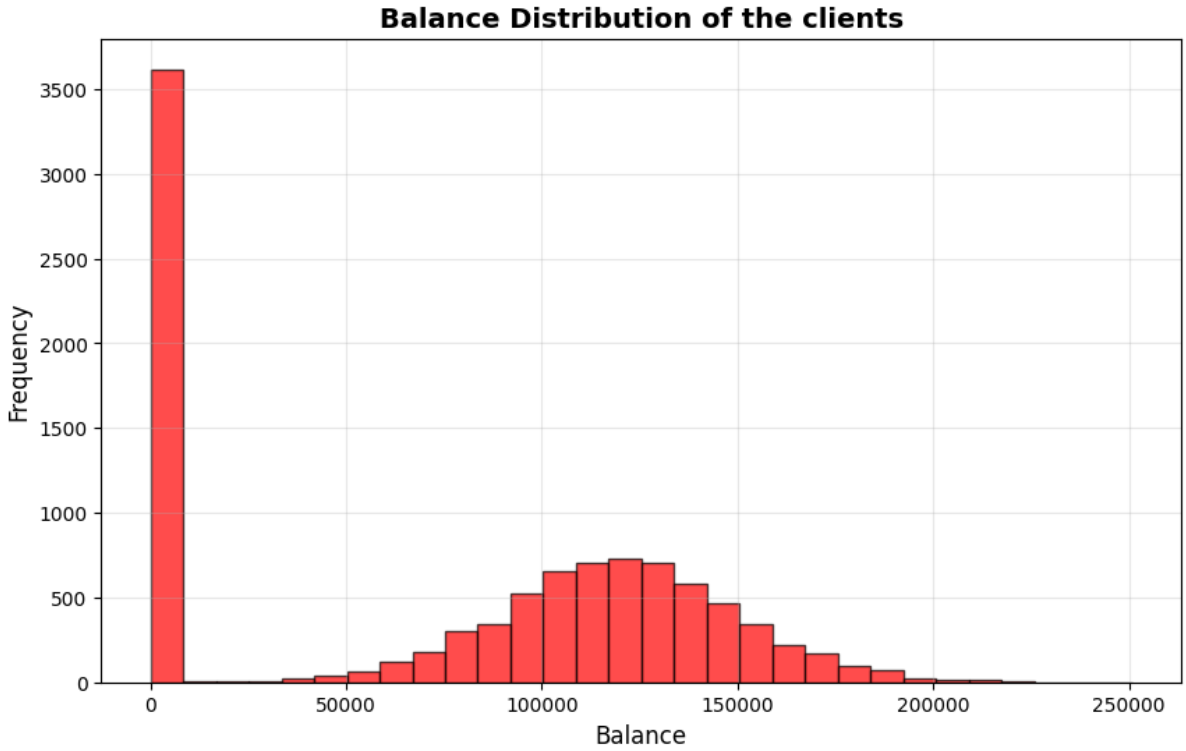
And 3 are **Binary:**

- HasCard
- IsActiveMember
- Exited (target feature)

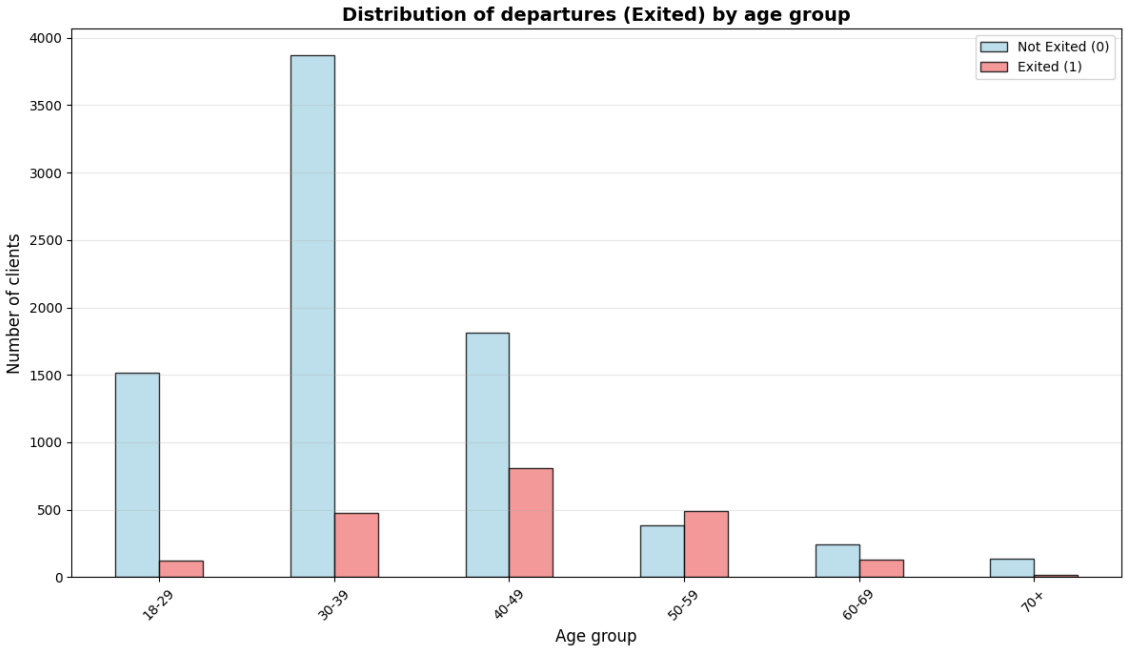
Data exploration, graphics and figures about our dataset

Before building the predictive model, a data exploration phase will be conducted to understand the structure, quality, and relationships within the dataset. This includes analyzing missing values, detecting outliers, and studying correlations between variables. The insights from this phase will guide feature selection and ensure that the model is built on reliable and meaningful data.

The following charts illustrate key insights from the exploratory data analysis, highlighting patterns and relationships that help understand customer behavior and churn tendencies.

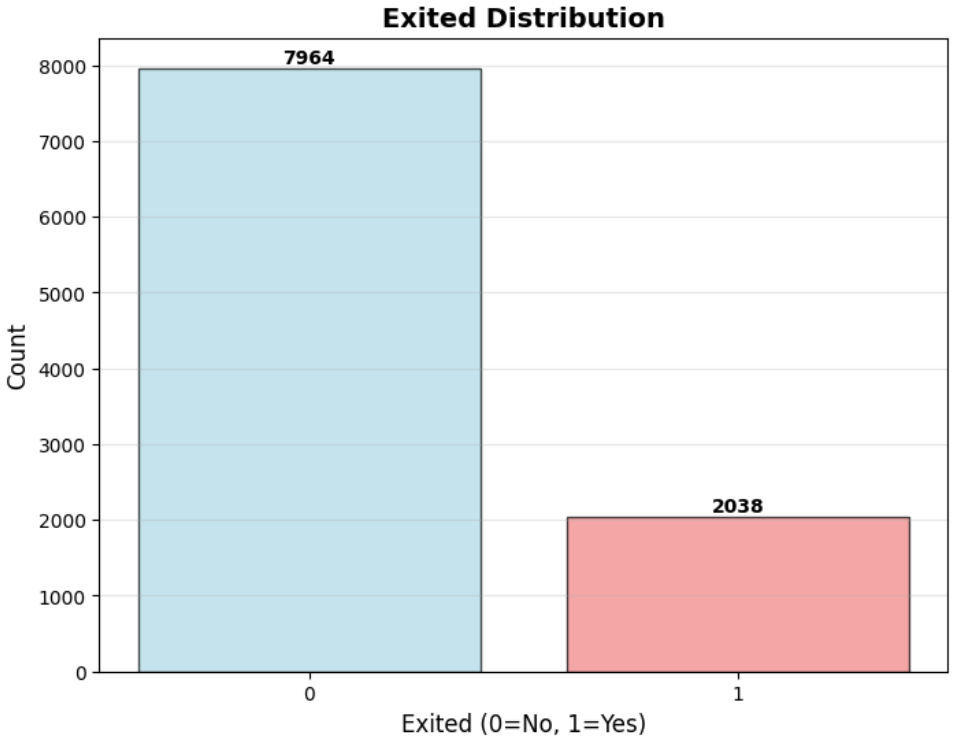


This histogram shows the distribution of client balances, revealing a strong concentration of customers with a zero balance. The remaining customers are distributed according to a normal distribution centered in 125,000€.
This plot is particularly relevant, as it highlights heterogeneity among clients and motivates feature engineering on the variable balance.



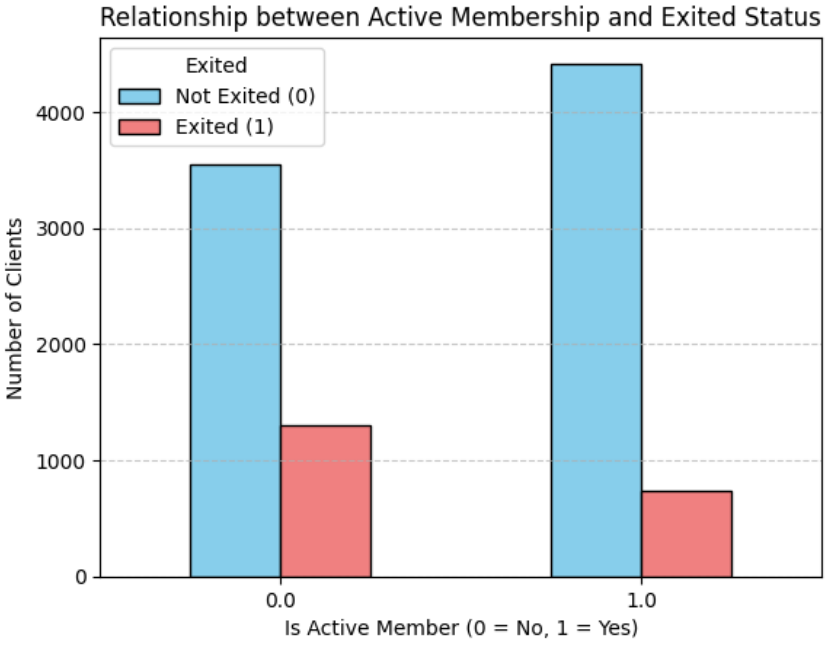
This chart shows that customers aged 40 to 59 are more likely to leave the bank compared to younger or older groups. This trend suggests that age is a significant predictor of churn, and it

will be useful to create features capturing age segments or non-linear effects in the predictive model.



This chart shows that around 20% of customers have left the bank, while the majority (about 80%) have stayed. This class imbalance is important to consider when training the predictive model, as it may require resampling techniques or adjusted evaluation metrics to ensure balanced performance.

We chose to use the SMOTE algorithm to handle this data imbalance, we implemented it a bit later.



This chart shows that inactive members are more likely to leave the bank than active ones. The difference is quite clear: customers who engage less with the bank's services tend to have a higher churn rate. This confirms that the customer activity level is a strong predictor of churn and should be included as an important feature in the predictive model.

Data preprocessing

Data cleaning

We have the chance that our dataset was clean. Indeed, we found that there were no missing values, no duplicates rows or entries and no inconsistencies.

Feature engineering

This was not asked for in the instructions, but it could be helpful for our future models to understand better the data and improve our results.

1. Remember that there are a lot of customers with a 0 balance. It might be interesting to create a new binary column "IsZeroBalance" that is '1' when the balance is 0 and '0' when the balance is more than 0.
2. Let's say customer A and B have a tenure equal to 5 years

   - Customer A (25 years old) is a very loyal customer. They have been a customer for a significant portion of their adult life.
   - Customer B (60 years old) is a very recent customer. They joined you only 5 years ago, perhaps after spending 35 years changing every 5 years of bank.

   These two customers are radically different, but the model, simply seeing Tenure = 5, will struggle to grasp this nuance. Thus, we created the "TenurePerAge" column.

3. Let's say two clients have a balance of 100,000€

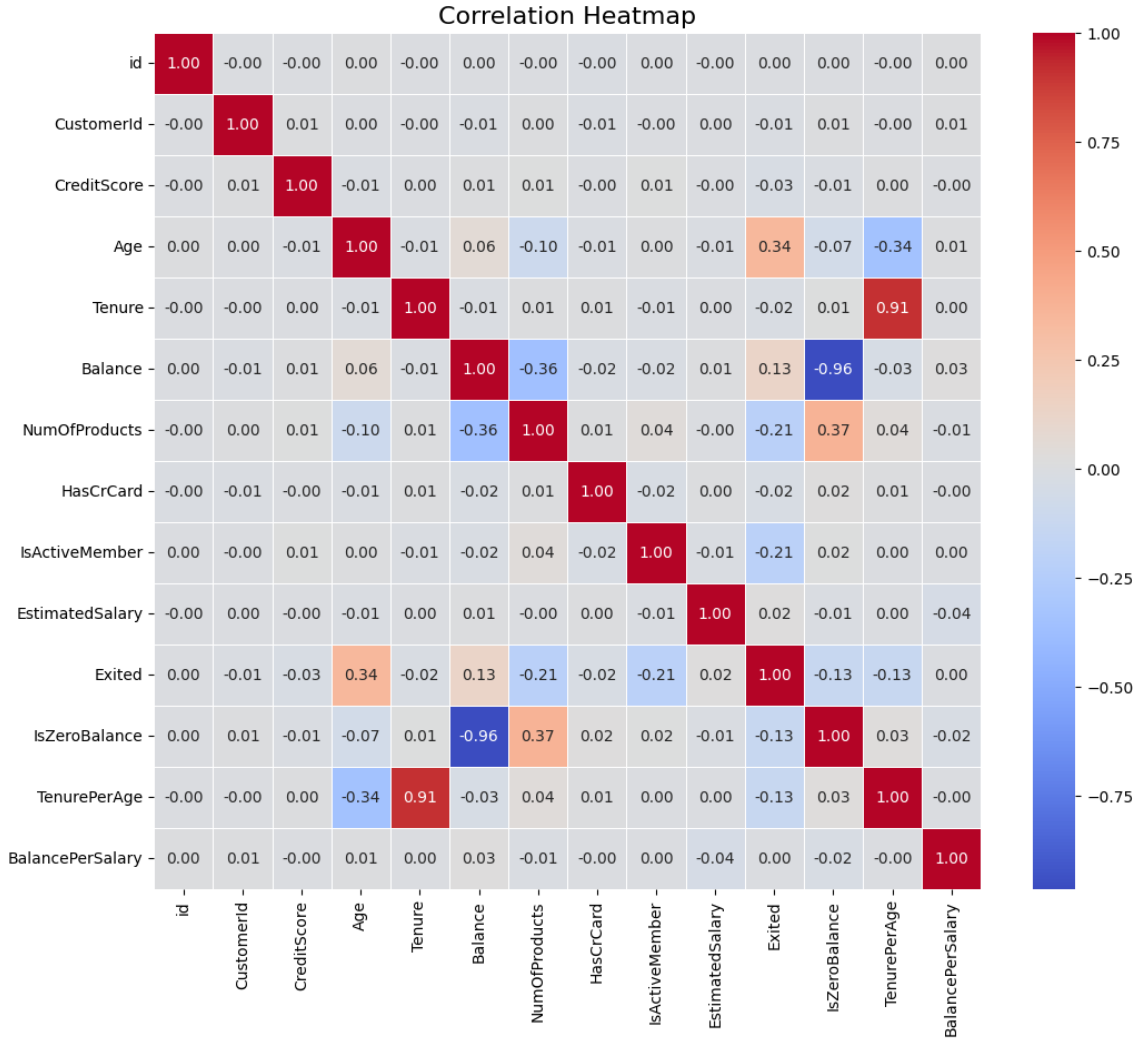
   - For Client A: This 100,000€ balance represents two full years of their salary. It's likely their entire savings. They are extremely invested in this bank. Losing this client would be a serious loss.
   - For Client B: This 100,000€ balance represents only six months of their salary. For this high-income client, it might just be a temporary account or spending money. They may be less focused on the service, but also much less loyal because their main assets are elsewhere.

   This is only a Hypothesis because our column is "EstimatedSalary" not "Salary", but we can still create it and have hope it helps!

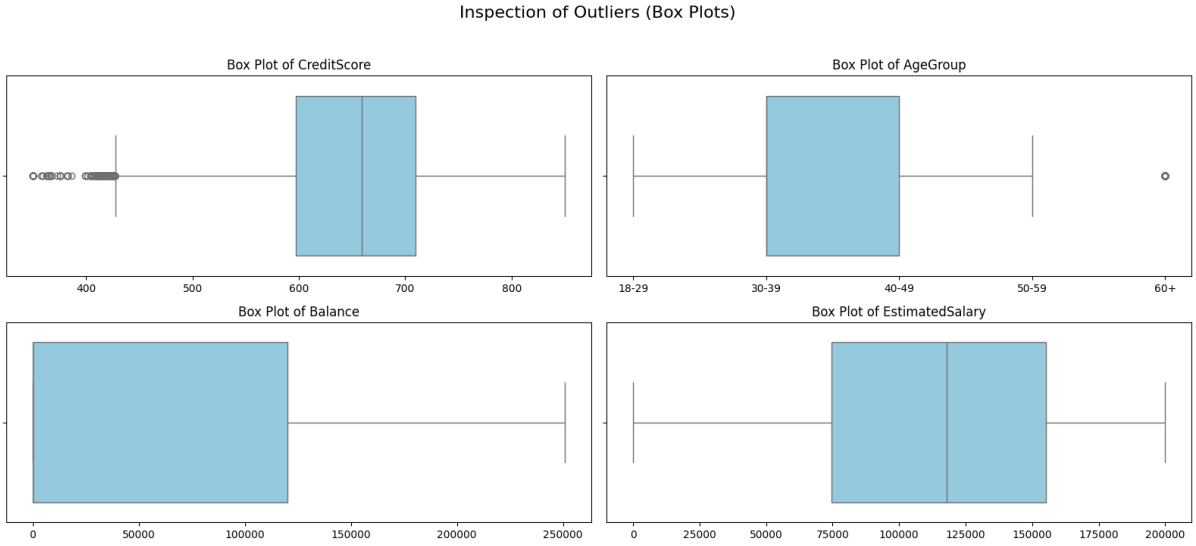
4. Finally, we modified the column "Age" to "AgeGroup" so the model can easily make some links between churning and the age.

Correlation Analysis

To see if our feature engineering worked or not. We can plot the correlation matrix.

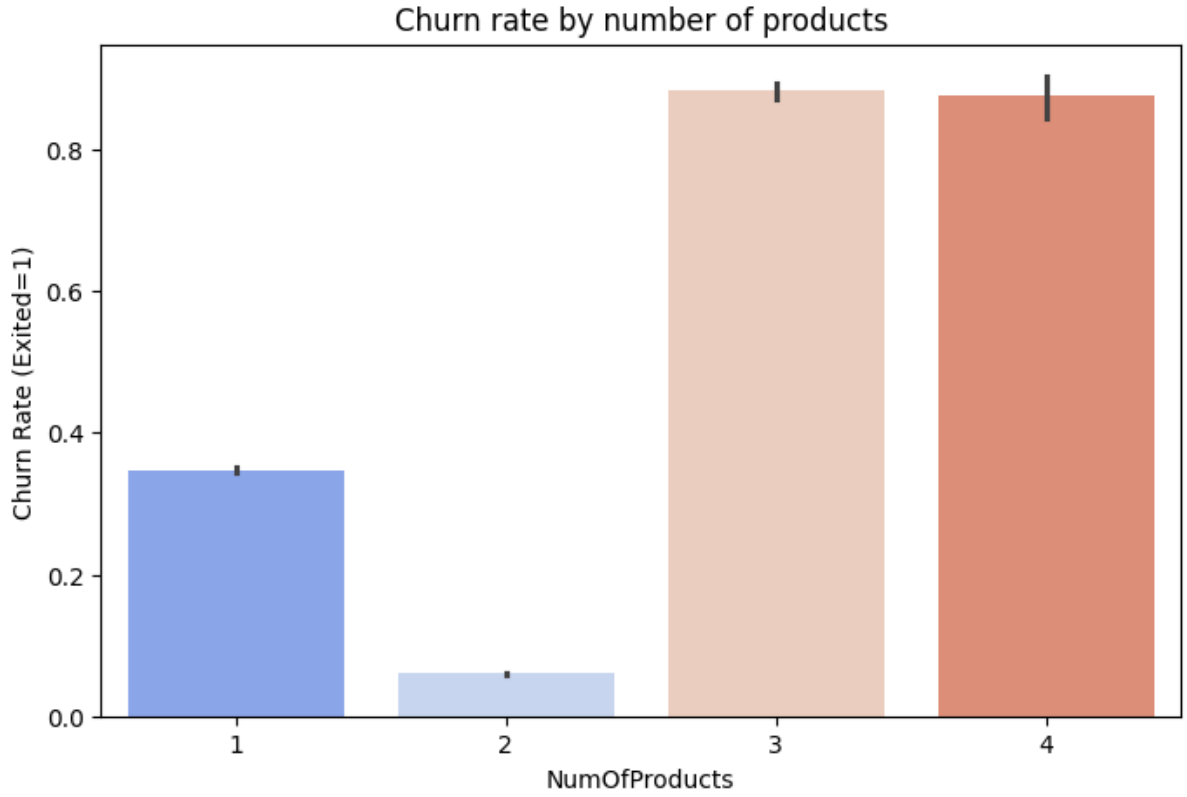


Outliers



We don't think that we need to remove the outliers because in this case the outliers are giving some insight into the data.

Encoding, Scaling and Splitting the dataset



Here we need to use the One-Hot encoding for these features, but also for those: 'Geography', 'Gender', 'AgeGroup'.

We have been very careful to take the following steps in the correct order.

1. Split the dataset into 3 parts:
    - Train set: 60%
    - Validation set: 20%
    - Test set: 20%
2. Apply the One-Hot Encoder and the StandardScaler on the X columns of all sets
3. Apply the SMOTE to the X_train_scaled and y_train

By following these steps, we ensure that no data leaks occur. But we also manage data imbalance here.

Formalization of the problem

Our project addresses a Binary Classification problem, as the objective is to predict whether a bank customer will leave or remain a customer.

The target variable is 'Exited', which is encoded as 0 ('Not Exited') and 1 ('Exited'). The dataset exhibits class imbalance, with the 'Not Exited' class (0) being the majority class. The Business Case prioritizes the accurate identification of the minority class (customers who will churn / 'Exited'), as this allows the bank to proactively intervene with retention strategies.

The standard metrics for optimizing an imbalanced model are F-scores or Area Under the Precision-Recall Curve (PR-AUC). The F-beta score (a generalized F-score) is the perfect choice because it allows us to explicitly weight Recall versus Precision:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

If $\beta = 1$ (F_1-score): Gives equal importance to Precision and Recall.

If $\beta < 1$: Gives more weight to Precision (minimizing False Positives).

If $\beta > 1$: Gives more weight to Recall (minimizing False Negatives).

Since our first model has a slightly lower Recall than a potential target, and we want to ensure that we capture the minority class (which usually means favoring Recall), the F_2-score is the best choice.

F_2-score places twice the weight on Recall compared to Precision, ensuring the Gridsearch focuses on finding the positive examples.
We want indeed to capture the minority class here because we want to maximize the detection of the client that has a high chance to churn to allow a bank to avoid it!

Presentation of our models

Every model implemented was implemented with the following steps:

1. Basic implementation: A basic implementation with some basic parameters.
2. Try to optimize the model: The main method we used for this was Gridsearch, which is a hyperparameter optimization technique that works by defining a list of potential settings for our model and systematically training it on every possible combination of these values. Then, it evaluates each combination using Cross-Validation, to identify the exact configuration that yields the highest performance metric.

Between each step we, of course, plot the results of the models and analyze them.

First Models

We started the binary classification with two very well-known methods: First, a logistic regression model that estimates the probability of a binary outcome (exited vs. stay) by fitting data to an S-shaped curve. In fact, it models the probability that an instance belongs to a specific class (0 or 1) using the logistic function (Sigmoid):

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0+\beta_1 x_1+\cdots+\beta_n x_n)}}$$

With this model we achieve an F2-score of 0.70.

Secondly, SVM (Support Vector Machine), a classifier that finds the optimal boundary (hyperplane) to separate “exited” from “non-exited” by maximizing the distance between them. It uses kernels to project data into higher dimensions to solve complex, non-linear separation problems.

Despite our efforts, this model was particularly overfitting and could only obtain a F2-score of 0.57. However, this model had a 100% recall, meaning that it catches all the clients that churned.

Dimension Reduction (PCA)

After that, we tried a dimensionality reduction with PCA (principal component analysis) that simplifies complex data by compressing many features into fewer components. It retains the most critical information (variance) while removing noise and redundancy.

We tried to reduce the problem into 2 Main Components, but the variance explained by this model was only ~40%. So, we could conclude that our Problem wasn’t a 2-dimensional problem.

After that we tried to reduce the dimension of the problem while keeping 95% of the variance explained but this model was not more efficient than our Logistic Regression model, with a

F2-score of 0.66 and 10 main components. It means that the 5% of information we rejected wasn't noise, but useful information and the components removed brought this information.

In our case, we concluded that our model couldn't be dimensionally reduced.

Advanced and Ensemble Models

Then, we moved to a bunch of more advanced and ensemble models for our prediction. These models were chosen either because we saw them in class (Machine Learning, Neural Networks & Deep Learning), or because our research led us to them.

The Scientific Paper we found[3] (from a Google Scholar search) tells us that Decision Tree, Random Forest and Gradient Boosting methods models have good results on problems like ours.

Decision Tree

It splits data into subsets based on the most significant differentiator in the input variables. It uses metrics like **Gini Impurity or Entropy** to determine the best split at each node. For example, a specific rule for the splitting can be "IF Age > 50 AND Balance = 0". After optimizing the Decision Tree, we could achieve a F2-score of 0.69.

Random Forest

To generalize it, we also coded a Random Forest classifier, an ensemble method that builds hundreds of Decision Trees independently on random subsets of data. It aggregates their predictions (majority vote) to create a robust model that reduces the risk of errors found in single trees. It is known as an ensemble model based on bagging, but we implemented voting and stacking classifiers as well. This model obtained a F2-score of 0.69.

Gradient Boosting

1. XGBoost

eXtreme Gradient Boosting, XGBoost, is an algorithm that builds trees sequentially rather than independently in Random Forest. Each new tree focuses specifically on correcting the errors made by the previous ones, making it highly accurate and efficient.

2. LightGBM

Light Gradient Boosting Machine, LightGBM, is a faster variant of gradient boosting that grows trees vertically (leaf-wise), a strategy that means growing the most promising branch rather than level-wise growth. It also groups data values into buckets (histograms), to bin continuous features, which speeds up training and reduces memory usage.

Our Gradient Boosting models both obtained an F2-score of approximately 0.69.

Voting Classifiers

A Voting Classifier model is a wrapper that combines the predictions of multiple different models (we tried many combinations of models), then, predicts the final class based on two kinds of selections, either a hard voting which predicts the class that gets the most votes from the estimator, or a soft voting which averages the probabilities predicted by each estimator.

We tried several options of combination of our precedent models and compared their F2-scores, the best Voting Classifier we created had a F2-score of 0.70.

Stacking Classifier

Stacking Classifier is more sophisticated than voting classifier, instead of simply voting, it uses a meta-learner (often a Logistic Regression) to learn how to best combine the predictions of the base models. Our best Stacking Classifier configuration obtained a F2-score of 0.68.

Neural Network

Finally, we were interested in a more common approach in deep learning, applying a Multi-Layer Perceptron model (MLP), creating an artificial neural network with layers of interconnected nodes. It learns by passing data forward and adjusting internal weights backward (backpropagation), capable of capturing highly abstract and complex non-linear patterns. After optimizing this advance model, we achieve a F2-Score of approximately 0.70.

We then tried a last time to do a Voting and a Stacking Classifier including this MLP model. Both models obtained a F2-score of approximately 0.70.

Obstacles

Main Obstacles

The main obstacle we encountered was overfitting. Almost all our models were overfitting. To encounter this, we did some research and modified the 'basic implementation' and the Gridsearches of each model with some "classic" parameters that have the reputation of being "anti-overfitting", in our code, we added some comments to explain the choice of these "anti-overfitting" Hyperparameters. We also modified our Gridsearches with "anti-overfitting" evaluation method such as Cross Validation, we tried Gridsearches with $\mathrm{cv} = 5$. However, another obstacle was that our Gridsearches were sometime very long to run, we were then forced to change the Cross Validation to $\mathrm{cv} = 3$ and reduce the number of Hyperparameters that we tested within our Gridsearches. Moreover, as said before, we also had a case of Data Imbalance. We handled that problem with the SMOTE Algorithm that we saw during our Lab Sessions.

Additional tests

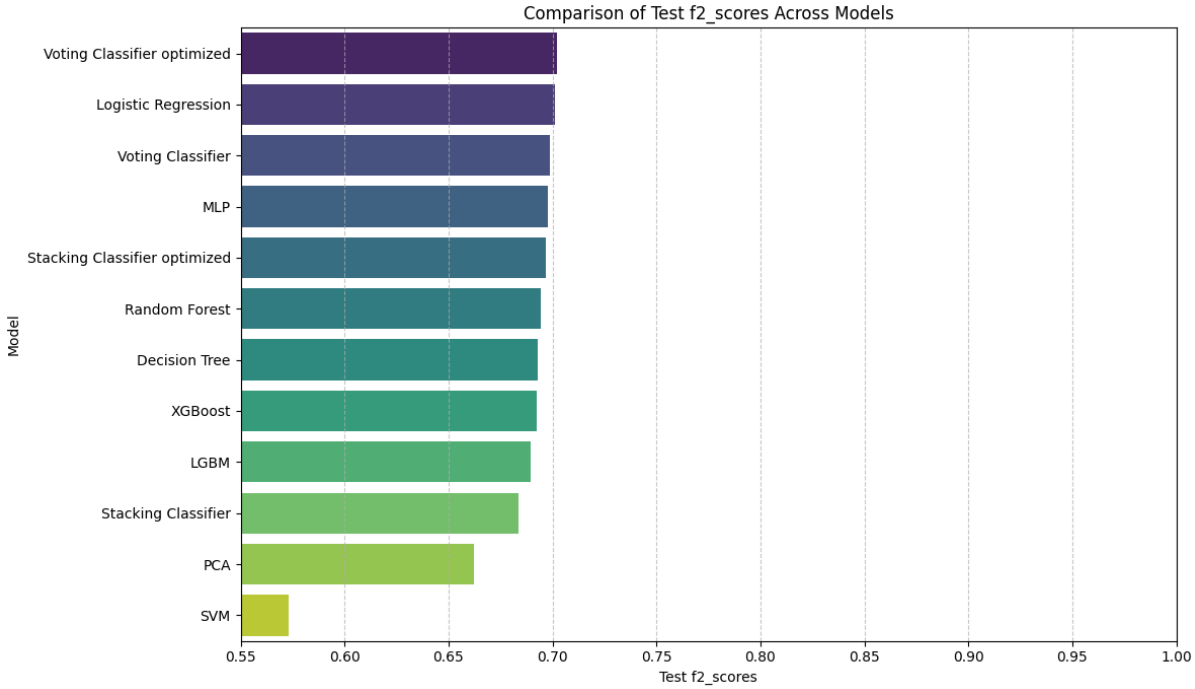
After, the presentation of our project during the last TD session. In order to definitively validate your work, our tutorial instructor suggested we take the following tests:

- Test to train our models without the SMOTE algorithm, to check if this is causing our models to hallucinate.
    - This test proved that we were right to use the SMOTE. Indeed, the F2-score of our decision tree, for example, went from 0.69 to 0.50.
- Test to train our models without the outliers (as said earlier we kept outliers because we think that it is not noise but information)
    - This test proved that we were right to not remove the outliers. Indeed, the F2-score of our decision tree, for example, went from 0.69 to 0.60.

This few tests, validated our assumptions made during the project.

Comparison of models results

Even if a lot of our models obtained similar F2-scores results, let's compare them.



We can see here that almost all our models stagnated around 0.70.

The SVM has a lower F2-score but a 100% recall and it is logical to have a less good PCA model because we explained earlier that our model couldn't be dimensionally reduced.

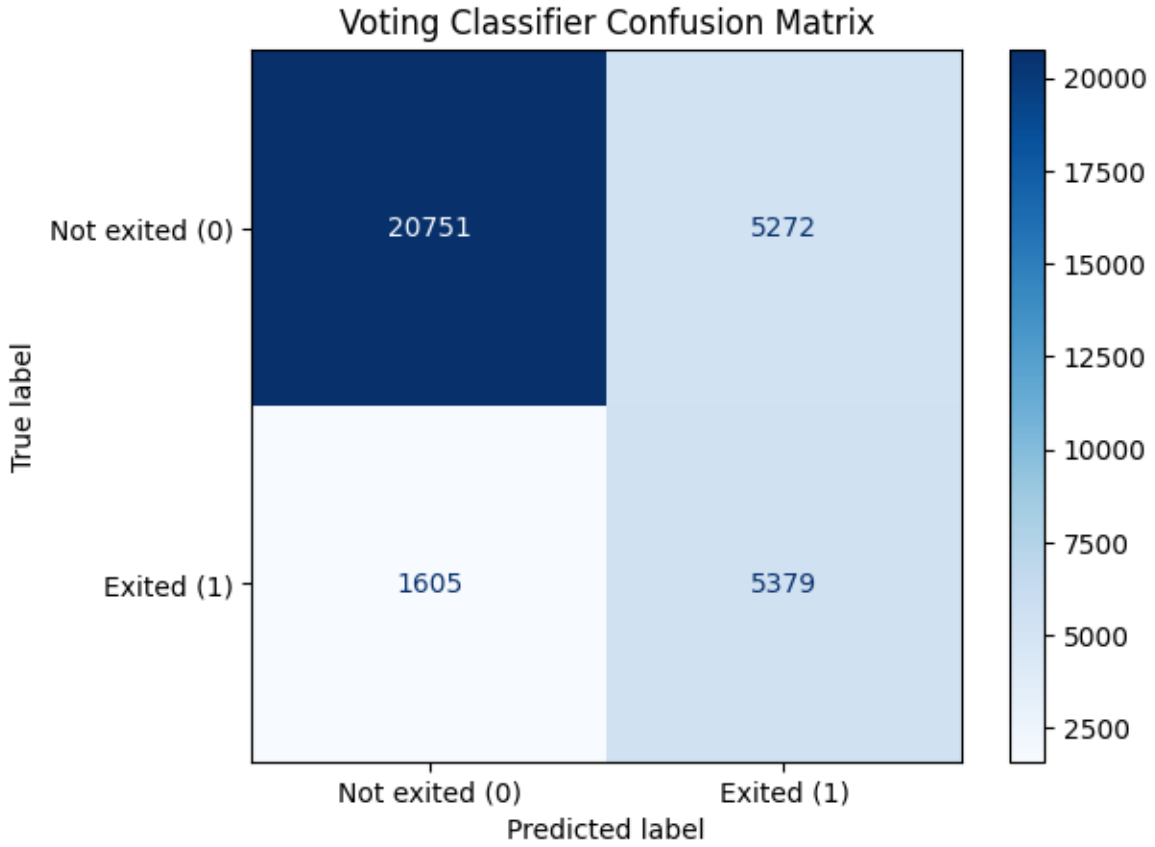
The Scientific Paper[3] also explained why the SVM models shows reduced performance in Binary Classification of Tabular data: it is due to its sensitivity to outliers.

Conclusion

We made a final test on our Test Data with our best model (Voting Classifier Optimized)

F2-score of our best model on the test data: 0.697.

The confusion matrix is the following:



Here can see that our model classifies 10 651 clients as “High Risk Clients” (i.e. predicts that they will churn). But only around 50% of them (51% to be precise) churn for real at the end. However, our model only misses 1 605 clients that churn, it means that our model catches around 75% (77% to be precise) of the clients that will exit the bank.

In a real-life scenario, this model is already helpful for a company, because even if it is not 100% accurate and a bit “paranoiac”, it helps the company to see 77% of the clients that churn. In the end, the company can try to make some new offers to try to keep all the clients that the model predicts as “High Risk”. If they do, they will reduce the number of churning clients by 77%.

Bibliography

1. Dataset source (Kaggle): https://www.kaggle.com/competitions/playground-series-s4e1/data?select=train.csv

2. Lecture: Neural Networks & Deep Learning: This is a lecture from the DIA master; among other things, it introduced us to the MLP model.
3. Scientific paper from research on Google Scholar: https://www.researchgate.net/publication/381254658_Machine_Learning_Techniques_for_Diabetes_Prediction_A_Comparative_Analysis